

LL0S 使用文档

目录

1. LL0S 介绍.....	2
2. 代码移植	2
2.1 目录结构	2
2.2 移植 OS.....	2
3. 修订日志	3

1. LLOS 介绍

LLOS 是一款轻量级的可以在支持 64bit 变量的 MCU 上运行的非实时资源管理系统，以事件驱动和状态机的方式实现线程管理，目的是取代裸机编程。系统内核提供简单的任务管理（事件状态机/协程）、任务间通信、软定时器、RTC、动态内存管理、设备驱动框架和指令解析功能。另外截至目前，已经配备了了好用且功能强大的按键驱动、LED/GPIO 驱动、CRC、环形 FIFO 等模块，其它模块还在不断地开发当中。

2. 代码移植

2.1 目录结构

```

└─ component
└─ drivers
└─ kernel

```

LLOS/kernel 路径下存放了 OS 的内核，其中有如下四个文件

```

└─ llos.c
└─ llos.h
└─ llos_conf.c
└─ llos_conf.h

```

对于内核代码，开发人员只需要配置 `llos_conf.h` 即可，除非您对代码非常熟悉，否则请不要修改其它文件。

LLOS/component 路径中存放了其它组件。这些模块的使用会在后面的章节进行介绍。

LLOS/drivers 路径中存放了其它驱动。这些模块的使用会在后面的章节进行介绍。

2.2 移植 OS

对于最基础 OS 的运行，kernel 路径下除了 `llos_conf.c` 的 3 个文件是必须的，在您的 IDE 中添加相关文件和头文件路径后，在您的代码中进行以下步骤的操作，在此之前注意将您的 IDE 编码格式设置为 UTF-8，否则可能出现中文注释乱码的情况。

1) 在初始化时调用 `LLOS_Init()`;

```

/**
 * @brief 初始化
 * @param[in] osCB: 回调注册
 * @param[in] cfigs: 配置
 */
void LLOS_Init(struct ll_init_CBs_t *osCB, struct ll_init_cfigs_t *cfigs);

```

2) 在 while(1) 中调用调用 LL0S_Loop();

```
/**
 * @brief OS处理函数，在死循环执行
 */
void LL0S_Loop(void);
```

3) 在定时器中断函数中调用 LL0S_Tick_Increase(x) 为 OS 提供时钟，可以使用 MCU 的滴答定时器等提供时钟；

```
/**
 * @brief OS节拍计数，在定时器中断中1ms调用一次
 */
void LL0S_Tick_Increase(void);
```

编译通过后，您已经完成了对 LL0S 的移植！具体的使用方法可以参考

- 1) 头文件注释及源代码；
- 2) 例程 demo；

3. 修订日志

注：版本号第一位更改表示架构级修改，第二位表示 API 不兼容修改，第三位 bug 修改或优化。

- 2026/07/30
发布 LL0S V3